

Name: Manahil Habib

Reg no: 2023-BSE-042

Section: 5B

Lab 11

Task 1 — Provider & Basic variable (variable precedence)

In Codespace create main.tf:

touch main.tf

```
@manahilhabib031 → /workspaces/Lab9 (main) $ touch main.tf
@manahilhabib031 → /workspaces/Lab9 (main) $ ls
MyED25519Key.pem  README.md  aws  awscli2.zip  main.tf  terraform.tfstate  terraform.tfstate.backup
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

Edit main.tf and add provider:

```
provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}
```

Command Prompt - gh codespace ssh

```
provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}
```

Initialize:

terraform init

```
@manahilhabib031 → /workspaces/Lab9 (main) $ terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.28.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

Define a variable and output:

```
variable "subnet_cidr_block" {
  type = string
}

output "subnet_cidr_block_output" {
  value = var.subnet_cidr_block
}
```

```
Command Prompt - gh codespace ssh
provider "aws" {
  shared_config_files      = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}
variable "subnet_cidr_block" {
  type = string
}

output "subnet_cidr_block_output" {
  value = var.subnet_cidr_block
}
```

Run apply (first time without defaults):

```
terraform apply -auto-approve
```

```
# it will prompt for the value of subnet_cidr_block
```

```

@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve
var.subnet_cidr_block
  Enter a value: 10.0.10.0/24

Changes to Outputs:
  ~ subnet_cidr_block_output = "yes" -> "10.0.10.0/24"

You can apply this plan to save these new output values to the Terraform state, without changing any real
infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
subnet_cidr_block_output = "10.0.10.0/24"
@manahilhabib031 → /workspaces/Lab9 (main) $ |

```

for variable input.

Add a default to the variable:

```

variable "subnet_cidr_block" {
  type    = string
  default = "10.0.0.0/24"
}

```

Run:

terraform apply -auto-approve

will use default without prompting

```

@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve

Changes to Outputs:
  - subnet_cidr_block_output = "10.0.10.0/24" -> null

You can apply this plan to save these new output values to the Terraform state, without changing any real
infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
@manahilhabib031 → /workspaces/Lab9 (main) $ |

```

Export environment variable in Codespace shell:

export TF_VAR_subnet_cidr_block=10.0.20.0/24

terraform apply -auto-approve

output should show environment value

```

@manahilhabib031 → /workspaces/Lab9 (main) $ export TF_VAR_subnet_cidr_block=10.0.20.0/24
terraform apply -auto-approve

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no
changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
@manahilhabib031 → /workspaces/Lab9 (main) $

```

Create terraform.tfvars overriding values:

touch terraform.tfvars

inside terraform.tfvars:

subnet_cidr_block = "10.0.30.0/24"

terraform apply -auto-approve

terraform.tfvars has priority over default and env

```
@manahilhabib031 → /workspaces/Lab9 (main) $ touch terraform.tfvars
@manahilhabib031 → /workspaces/Lab9 (main) $ vim terraform.tfvars
"terraform.tfvars" 1L, 35B written @manahilhabib031 → /workspaces/Lab9 (main) $ cat terraform.tfvars
subnet_cidr_block = "10.0.30.0/24"
@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve -var "subnet_cidr_block=10.0.40.0/24"

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

Show and unset env var:

printenv | grep TF_VAR_

unset TF_VAR_subnet_cidr_block

printenv | grep TF_VAR_

```
@manahilhabib031 → /workspaces/Lab9 (main) $ printenv | grep TF_VAR_
TF_VAR_subnet_cidr_block=10.0.20.0/24
@manahilhabib031 → /workspaces/Lab9 (main) $ unset TF_VAR_subnet_cidr_block
@manahilhabib031 → /workspaces/Lab9 (main) $ printenv | grep TF_VAR_
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

Task 2 — Variable validation & sensitive / ephemeral variables

Replace subnet_cidr_block variable with this (validation included):

variable "subnet_cidr_block" {

```

type      = string
default   = ""

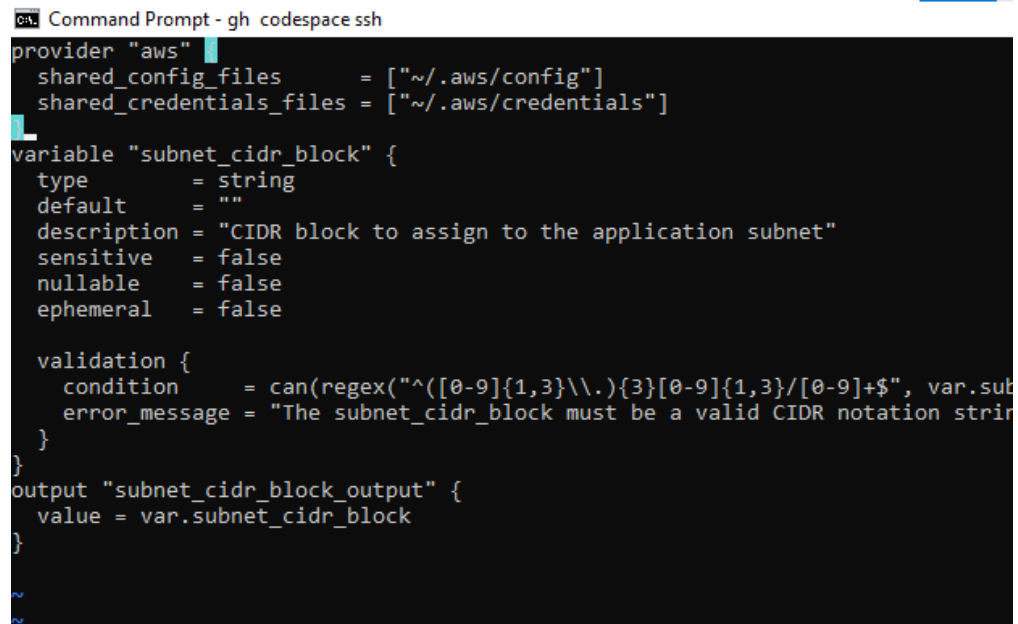
description = "CIDR block to assign to the application subnet"

sensitive  = false
nullable   = false
ephemeral  = false

validation {
    condition = can(regex("^[0-9]{1,3}\.){3}[0-9]{1,3}/[0-9]+$", var.subnet_cidr_block))

    error_message = "The subnet_cidr_block must be a valid CIDR notation string, such as
10.0.0.0/24."
}
}

```



The screenshot shows a terminal window titled "Command Prompt - gh codespace ssh". It displays the following Terraform configuration code:

```

provider "aws" {
  shared_config_files = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

variable "subnet_cidr_block" {
  type      = string
  default   = ""
  description = "CIDR block to assign to the application subnet"
  sensitive  = false
  nullable   = false
  ephemeral  = false

  validation {
    condition = can(regex("^[0-9]{1,3}\.){3}[0-9]{1,3}/[0-9]+$", var.subnet_cidr_block))
    error_message = "The subnet_cidr_block must be a valid CIDR notation string, such as 10.0.0.0/24."
  }
}

output "subnet_cidr_block_output" {
  value = var.subnet_cidr_block
}

```

Test validation failure:

```
terraform apply -auto-approve -var "subnet_cidr_block=10.0.0"
```

should show validation error

```

@manahilhabib031 → /workspaces/Lab9 (main) $ printenv | grep TF_VAR_
@manahilhabib031 → /workspaces/Lab9 (main) $ vim main.tf
@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve -var "subnet_cidr_block=10.0.0"

Error: Invalid value for variable

  on main.tf line 12:
 12: variable "subnet_cidr_block" {
    |
    |   var.subnet_cidr_block is "10.0.0"
    |
    | The subnet_cidr_block must be a valid CIDR notation string, such as 10.0.0.0/24.
    | This was checked by the validation rule at main.tf:20,3-13.
    |
Error: Invalid value for variable

  on main.tf line 29:
 29: variable "api_session_token" {
    |
    | The API session token must be at least 20 characters and contain only letters, numbers, hyphens, or
    | underscores.
    | This was checked by the validation rule at main.tf:37,3-13.
@manahilhabib031 → /workspaces/Lab9 (main) $ |

```

Create a sensitive variable `api_session_token` and output (sensitive):

```

variable "api_session_token" {
  type      = string
  default   = ""
  description = "Short-lived API session token used during apply operations"
  sensitive  = true
  nullable  = false
  ephemeral  = false
  validation {
    condition     = can(regex("^[A-Za-z0-9-_{20,}$", var.api_session_token))
    error_message = "The API session token must be at least 20 characters and contain only
letters, numbers, hyphens, or underscores."
  }
}

output "api_session_token_output" {
  value     = var.api_session_token
  sensitive = true
}

```

```

Command Prompt - gh codespace ssh
provider "aws" {
  shared_config_files = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}
variable "subnet_cidr_block" {
  type = string
  default = ""
  description = "CIDR block to assign to the application subnet"
  sensitive = false
  nullable = false
  ephemeral = false

  validation {
    condition = can(regex("^[0-9]{1,3}\\.[0-9]{1,3}/[0-9]+$", var.subnet_cidr_block))
    error_message = "The subnet_cidr_block must be a valid CIDR notation string, such as 10.0.30.0/24"
  }
}
variable "api_session_token" {
  type = string
  default = ""
  description = "Short-lived API session token used during apply operations"
  sensitive = true
  nullable = false
  ephemeral = false

  validation {
    condition = can(regex("^[A-Za-z0-9-]{20,}$", var.api_session_token))
    error_message = "The API session token must be at least 20 characters and contain only alphanumeric characters and hyphens"
  }
}
output "api_session_token_output" {
  value = var.api_session_token
  sensitive = true
}
output "subnet_cidr_block_output" {
  value = var.subnet_cidr_block
}

```

Run with -var to observe sensitive output behavior:

terraform apply -auto-approve -var "api_session_token=my_API_session_Token"

output will be marked sensitive; check terraform.tfstate for outputs

```

@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve -var "api_session_token=my_API_session_Token"

Changes to Outputs:
+ api_session_token_output = (sensitive value)
+ subnet_cidr_block_output = "10.0.30.0/24"

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
api_session_token_output = <sensitive>
subnet_cidr_block_output = "10.0.30.0/24"
@manahilhabib031 → /workspaces/Lab9 (main) $ |

```

Check terraform.state for the sensitive output:

You should find an outputs section similar to:

```
"api_session_token_output": {
  "value": "my_API_session_Token",
  "type": "string",
  "sensitive": true
}
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ cat terraform.tfstate | grep api_session_token_output -A 5
  "api_session_token_output": {
    "value": "my_API_session_Token",
    "type": "string",
    "sensitive": true
  },
  "subnet_cidr_block_output": {
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

Make variable ephemeral to hide from state:

```
variable "api_session_token" {
  ...
  ephemeral = true
  ...
}
```

```
}
variable "api_session_token" {
  type          = string
  default       = ""
  description   = "Short-lived API session token used for authentication"
  sensitive     = true
  nullable     = false
  ephemeral    = true

  validation {
    condition     = can(regex("^[A-Za-z0-9-]{20,}$", var.api_session_token))
    error_message = "The API session token must be 20 characters long and contain only alphanumeric characters and hyphens"
  }
}
```

Set default to test local default:

```
variable "api_session_token" {
  default = "my_API_session_Token"
  sensitive = true
}
```

```
ephemeral = false
```

```
...
```

```
}
```

```
terraform apply -auto-approve
```

```
# works and stored in state (but output remains sensitive)
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve -var "api_session_token=my_API_session_Token"

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
api_session_token_output = <sensitive>
subnet_cidr_block_output = "10.0.30.0/24"
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

Task 3 — Project-level variables, locals, and outputs

Add variables to main.tf:

```
variable "environment" {}
```

```
variable "project_name" {}
```

```
variable "primary_subnet_id" {}
```

```
variable "subnet_count" {}
```

```
variable "monitoring" {}
```

```
value = var.subnet_cidr_block
}
variable "environment" {}
variable "project_name" {}
variable "primary_subnet_id" {}
variable "subnet_count" {}
variable "monitoring" {}
```

Populate terraform.tfvars after discovering actual subnet id for availability zone me-central-1a:

```
aws ec2 describe-subnets \
```

```
--filters "Name=availability-zone,Values=me-central-1a" \
```

```
--query "Subnets[].SubnetId" \
```

```
--output text
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-subnets \
  --filters "Name=availability-zone,Values=me-central-1a" \
  --query "Subnets[].SubnetId" \
  --output text
subnet-062560d61e753ef93      subnet-0f6a39ffa160d4732
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

Set values in terraform.tfvars:

```
environment = "dev"
```

```
project_name = "lab_work"
```

```
primary_subnet_id = "<subnet-id-of-me-central-1a>"
```

```
subnet_count = 3
```

```
monitoring = true
```

CA: Command Prompt - gh codespace ssh

```
environment = "dev"
project_name = "lab_work"
primary_subnet_id = "subnet-0bf5c35a7b6277b37"
subnet_count = 3
monitoring = true
```

Create locals.tf with:

```
locals {
```

```
  resource_name = "${var.project_name}-${var.environment}"
```

```
  primary_public_subnet = var.primary_subnet_id
```

```
  subnet_count      = var.subnet_count
```

```
  is_production      = var.environment == "prod"
```

```
  monitoring_enabled = var.monitoring || local.is_production
```

```
}
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ touch terraform.tfvars
@manahilhabib031 → /workspaces/Lab9 (main) $ vim terraform.tfvars
@manahilhabib031 → /workspaces/Lab9 (main) $ touch locals.tf
@manahilhabib031 → /workspaces/Lab9 (main) $ vim locals.tf
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

Command Prompt - gh codespace ssh

```
locals {
  resource_name = "${var.project_name}-${var.environment}"
  primary_public_subnet = var.primary_subnet_id
  subnet_count      = var.subnet_count
  is_production      = var.environment == "prod"
  monitoring_enabled = var.monitoring || local.is_production
}
```

Add outputs to main.tf:

```
output "resource_name" {
  value = local.resource_name
}

output "primary_public_subnet" {
  value = local.primary_public_subnet
}

output "subnet_count" {
  value = local.subnet_count
}

output "is_production" {
  value = local.is_production
}

output "monitoring_enabled" {
  value = local.monitoring_enabled
}
```

Run:

```
terraform apply -auto-approve
```

will show all the output values

```

@manahilhabib031 → /workspaces/Lab9 (main) $ terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.28.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve

Changes to Outputs:
- api_session_token_output = (sensitive value) -> null
+ is_production            = false
+ monitoring_enabled       = true
+ primary_public_subnet    = "<subnet-id-of-me-central-la>"
+ resource_name            = "lab_work-dev"
- subnet_cidr_block_output = "10.0.30.0/24" -> null
+ subnet_count            = 3

You can apply this plan to save these new output values to the Terraform state, without changing any real
infrastructure.

Warning: Value for undeclared variable

The root module does not declare a variable named "subnet_cidr_block" but a value was found in file
"terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR_... environment variables to provide certain "global" settings to all
configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings
option.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

is_production = false
monitoring_enabled = true
primary_public_subnet = "<subnet-id-of-me-central-la>"
resource_name = "lab_work-dev"
subnet_count = 3
@manahilhabib031 → /workspaces/Lab9 (main) $ |

```

Task 4 — Maps and Objects

Map variable in main.tf:

```

variable "tags" {
  type = map(string)
}

output "tags" {
  value = var.tags
}

```

```

}
# Map variable definition
variable "tags" {
  type = map(string)
}

# Output for tags
output "tags" {
  value = var.tags
}

~
~
~
~
:~

```

In terraform.tfvars:

```
tags = {
  Environment = "dev"
  Project     = "sample-app"
  Owner       = "platform-team"
}
```

```
tags = {
  Environment = "dev"
  Project     = "sample-app"
  Owner       = "platform-team"
}
```

Run:

```
terraform apply -auto-approve
```

```
# check tags output
```

```

@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve

Changes to Outputs:
+ tags = {
+   Environment = "dev"
+   Owner       = "platform-team"
+   Project     = "sample-app"
+ }

You can apply this plan to save these new output values to the Terraform state, without changing any real
infrastructure.

Warning: Value for undeclared variable

The root module does not declare a variable named "subnet_cidr_block" but a value was found in file
"terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR... environment variables to provide certain "global" settings to all
configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings
option.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

is_production = false
monitoring_enabled = true
primary_public_subnet = "<subnet-id-of-me-central-la>"
resource_name = "lab_work-dev"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner"       = "platform-team"
  "Project"     = "sample-app"
})
@manahilhabib031 → /workspaces/Lab9 (main) $ |

```

Define object variable:

```

variable "server_config" {
  type = object({
    name          = string
    instance_type = string
    monitoring     = bool
    storage_gb     = number
    backup_enabled = bool
  })
}

output "server_config" {
  value = var.server_config
}

```

```

# Output for tags
output "tags" {
  value = var.tags
}

# Object variable definition
variable "server_config" {
  type = object({
    name           = string
    instance_type  = string
    monitoring     = bool
    storage_gb     = number
    backup_enabled = bool
  })
}

# Output for server_config
output "server_config" {
  value = var.server_config
}

-- REPLACE --

```

In terraform.tfvars:

```

server_config = {
  name           = "web-server"
  instance_type  = "t3.micro"
  monitoring     = true
  storage_gb     = 20
  backup_enabled = false
}

```

```

server_config = {
  name           = "web-server"
  instance_type  = "t3.micro"
  monitoring     = true
  storage_gb     = 20
  backup_enabled = false
}

```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve
```

Changes to Outputs:

```
+ server_config      = {
+   backup_enabled = false
+   instance_type  = "t3.micro"
+   monitoring     = true
+   name           = "web-server"
+   storage_gb     = 20
+ }
```

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Warning: Value for undeclared variable

The root module does not declare a variable named "subnet_cidr_block" but a value was found in file "terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR_... environment variables to provide certain "global" settings to all configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings option.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

```
is_production = false
monitoring_enabled = true
primary_public_subnet = "<subnet-id-of-me-central-1a>"
resource_name = "lab_work-dev"
server_config = {
  "backup_enabled" = false
  "instance_type" = "t3.micro"
  "monitoring" = true
  "name" = "web-server"
  "storage_gb" = 20
}
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner" = "platform-team"
  "Project" = "sample-app"
})
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

Task 5 — Collections: list, tuple, set & mutation via locals

In this task you will define collection variables (list, tuple, set), observe their behavior, then perform mutations via locals and compare results.

Feature List Tuple Set

Order preserved ✓ ✓ ✗

Allows duplicates ✓ ✓ ✗

Mixed types ✗ ✓ ✗

Fixed size ✗ ✓ ✗

Mutable ✓ ✗ ✓

Best for Flexible sequences Structured records Unique collections

In main.tf define:

```
variable "server_names" {  
  type = list(string)  
  default = ["web-2", "web-1", "web-2"]  
}  
  
variable "server_metadata" {  
  type = tuple([string, number, bool])  
  default = ["web-1", 4, true]  
}  
  
variable "availability_zones" {  
  type = set(string)  
  default = ["me-central-1b", "me-central-1a", "me-central-1b"]  
}  
  
output "compare_collections" {  
  value = {  
    list_example = var.server_names  
    tuple_example = var.server_metadata  
    set_example = var.availability_zones  
  }  
}
```

```
ca: Command Prompt - gh codespace ssh
provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

#_list variable
variable "server_names" {
  type    = list(string)
  default = ["web-2", "web-1", "web-2"]
}

# Tuple variable
variable "server_metadata" {
  type    = tuple([string, number, bool])
  default = ["web-1", 4, true]
}

# Set variable
variable "availability_zones" {
  type    = set(string)
  default = ["me-central-1b", "me-central-1a", "me-central-1b"]
}

# Output to compare collections
output "compare_collections" {
  value = {
    list_example    = var.server_names
    tuple_example   = var.server_metadata
    set_example     = var.availability_zones
  }
}
```

Run:

terraform apply -auto-approve

Observe the output — note ordering, duplicates, and that sets are displayed without duplicates and order is not guaranteed.

```
@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve
```

Changes to Outputs:

```
+ compare_collections = {  
  + list_example = [  
    + "web-2",  
    + "web-1",  
    + "web-2",  
  ]  
  + set_example = [  
    + "me-central-1a",  
    + "me-central-1b",  
  ]  
  + tuple_example = [  
    + "web-1",  
    + 4,  
    + true,  
  ]  
}
```

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Warning: Value for undeclared variable

The root module does not declare a variable named "subnet_cidr_block" but a value was found in file "terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR_... environment variables to provide certain "global" settings to all configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings option.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

```
compare_collections = {  
  "list_example" = tolist([  
    "web-2",  
    "web-1",  
    "web-2",  
  ])  
  "set_example" = toset([  
    "me-central-1a",  
    "me-central-1b",  
  ])  
  "tuple_example" = [  
    "web-1",  
    4,  
    true,  
  ]  
}
```

In locals.tf add mutations: Create or edit locals.tf and add the following locals to demonstrate mutation behavior. Note: tuples are immutable in Terraform's type system, but many operations convert them to lists for evaluation.

```
locals {
```

```
  mutated_list = setunion(var.server_names, ["web-3"])
```

```
  mutated_tuple = setunion(var.server_metadata, ["web-2"])
```

```
  mutated_set = setunion(var.availability_zones, ["me-central-1c"])
```

```
}
```

CA: Command Prompt - gh codespace ssh

```
locals {  
  mutated_list = setunion(var.server_names, ["web-3"])  
  mutated_tuple = setunion(var.server_metadata, ["web-2"])  
  mutated_set = setunion(var.availability_zones, ["me-central-1c"])  
}
```

Add comparison output in main.tf:

```
output "mutation_comparison" {  
  
  value = {  
  
    original_tuple = var.server_metadata  
    mutated_tuple = local.mutated_tuple  
  
  }  
  
}
```

```
}  
}  
output "mutation_comparison" {  
  value = {  
    original_tuple = var.server_metadata  
    mutated_tuple = local.mutated_tuple  
  }  
}
```

Run:

terraform apply -auto-approve

Observe the difference between the original tuple and the mutated result. Note that Terraform may represent the mutated tuple as a list and the ordering/duplicates behavior will reflect setunion semantics.

```

@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve

Changes to Outputs:
+ mutation_comparison = {
+   mutated_tuple = [
+     "4",
+     "true",
+     "web-1",
+     "web-2",
+   ]
+   original_tuple = [
+     "web-1",
+     4,
+     true,
+   ]
+ }

You can apply this plan to save these new output values to the Terraform state, without changing any real
infrastructure.

Warning: Value for undeclared variable

The root module does not declare a variable named "subnet_cidr_block" but a value was found in file
"terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR_... environment variables to provide certain "global" settings to all
configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings
option.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
compare_collections = {
  "list_example" = tolist([
    "web-2",
    "web-1",
    "web-2",
  ])
  "set_example" = toset([
    "me-central-1a",
    "me-central-1b",
  ])
  "tuple_example" = [
    "web-1",
    4,
    true,
  ]
}
is_production = false

```

Task 6 — Null, any type & dynamic values

Null variable:

```

variable "optional_tag" {
  type      = string
  description = "A tag that may or may not be provided"
  default   = null
}

```

```

}

variable "optional_tag" {
  type      = string
  description = "A tag that may or may not be provided"
  default   = null
}

```

Merge tags in locals.tf:

```
locals {  
  server_tags = merge(  
    { Name = "web-server" },  
    var.optional_tag != null ? { Custom = var.optional_tag } : {}  
  )  
}
```

```
locals {  
  server_tags = merge(  
    { Name = "web-server" },  
    var.optional_tag != null ? { Custom = var.optional_tag } : {}  
  )  
}
```

Output:

```
output "optional_tag" {  
  value = local.server_tags  
}
```

```
output "optional_tag" {  
  value = local.server_tags  
}  
  
-- INSERT --
```

Run:

```
terraform apply -auto-approve
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve
```

Changes to Outputs:

```
- compare_collections = {
  - list_example = [
    - "web-2",
    - "web-1",
    - "web-2",
  ]
  - set_example = [
    - "me-central-1a",
    - "me-central-1b",
  ]
  - tuple_example = [
    - "web-1",
    - 4,
    - true,
  ]
} -> null
- is_production      = false -> null
- monitoring_enabled = true  -> null
- mutation_comparison = {
```

```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
```

Outputs:

```
optional_tag = {
  "Name" = "web-server"
}
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

Add in terraform.tfvars:

```
optional_tag = "dev"
```

```
terraform apply -auto-approve
```

observe that Custom tag appears

```
optional_tag = "dev"
```

```

@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve

Changes to Outputs:
  ~ optional_tag = {
    + Custom = "dev"
    # (1 unchanged attribute hidden)
  }

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Warning: Value for undeclared variable

The root module does not declare a variable named "monitoring" but a value was found in file
"terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR_... environment variables to provide certain "global" settings to all
configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings
option.

Warning: Value for undeclared variable

The root module does not declare a variable named "subnet_count" but a value was found in file
"terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR_... environment variables to provide certain "global" settings to all
configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings
option.

Warning: Values for undeclared variables

In addition to the other similar warnings shown, 6 other variable(s) defined without being declared.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
optional_tag = {
  "Custom" = "dev"
  "Name" = "web-server"
}
@manahilhabib031 → /workspaces/Lab9 (main) $ |

```

Any type variable:

```

variable "dynamic_value" {
  type      = any
  description = "A variable that can accept any data type"
  default   = null
}

output "value_received" {
  value = var.dynamic_value
}

```

```
variable "dynamic_value" {  
  type      = any  
  description = "A variable that can accept any data type"  
  default   = null  
}  
  
output "value_received" {  
  value = var.dynamic_value  
}  
  
-- INSERT --
```

Now test the dynamic (any) variable with different types. For each change, update terraform.tfvars with the specified value and run terraform apply -auto-approve, then capture the output.

a) String

In terraform.tfvars:

dynamic_value = "hello"

Run:

terraform apply -auto-approve

```

@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve

Changes to Outputs:
  + value_received = "hello"

You can apply this plan to save these new output values to the Terraform state, without changing any real
infrastructure.

Warning: Value for undeclared variable

The root module does not declare a variable named "subnet_count" but a value was found in file
"terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR_... environment variables to provide certain "global" settings to all
configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings
option.

Warning: Value for undeclared variable

The root module does not declare a variable named "monitoring" but a value was found in file
"terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR_... environment variables to provide certain "global" settings to all
configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings
option.

Warning: Values for undeclared variables

In addition to the other similar warnings shown, 6 other variable(s) defined without being declared.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

optional_tag = {
  "Custom" = "dev"
  "Name" = "web-server"
}
value_received = "hello"
@manahilhabib031 → /workspaces/Lab9 (main) $ |

```

b) Number

Change terraform.tfvars:

dynamic_value = 42

Run:

terraform apply -auto-approve

```

@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve

Changes to Outputs:
  ~ value_received = "hello" -> 42

You can apply this plan to save these new output values to the Terraform state, without changing any real
infrastructure.

Warning: Value for undeclared variable

The root module does not declare a variable named "server_config" but a value was found in file
"terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR_... environment variables to provide certain "global" settings to all
configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings
option.

Warning: Value for undeclared variable

The root module does not declare a variable named "subnet_cidr_block" but a value was found in file
"terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR_... environment variables to provide certain "global" settings to all
configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings
option.

Warning: Values for undeclared variables

In addition to the other similar warnings shown, 6 other variable(s) defined without being declared.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
optional_tag = {
  "Custom" = "dev"
  "Name" = "web-server"
}
value_received = 42
@manahilhabib031 → /workspaces/Lab9 (main) $ |

```

c) List

Change terraform.tfvars:

dynamic_value = ["a", "b", "c"]

Run:

terraform apply -auto-approve

```

@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve
Changes to Outputs:
  ~ value_received = 42 -> [
    + "a",
    + "b",
    + "c",
  ]

You can apply this plan to save these new output values to the Terraform state, without changing any real
infrastructure.

Warning: Value for undeclared variable

The root module does not declare a variable named "subnet_count" but a value was found in file
"terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR... environment variables to provide certain "global" settings to all
configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings
option.

Warning: Value for undeclared variable

The root module does not declare a variable named "server_config" but a value was found in file
"terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR... environment variables to provide certain "global" settings to all
configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings
option.

Warning: Values for undeclared variables

In addition to the other similar warnings shown, 6 other variable(s) defined without being declared.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
optional_tag = {
  "Custom" = "dev"
  "Name" = "web-server"
}
value_received = [
  "a",
  "b",
  "c",
]
@manahilhabib031 → /workspaces/Lab9 (main) $ |

```

d) Map / Object

Change terraform.tfvars:

```
dynamic_value = {
```

```
  name = "server"
```

```
  cpu = 4
```

```
}
```

Run:

```
terraform apply -auto-approve
```

```

@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve

Changes to Outputs:
  ~ value_received = [
    - "a",
    - "b",
    - "c",
  ] -> {
    + cpu = 4
    + name = "server"
  }

You can apply this plan to save these new output values to the Terraform state, without changing any real
infrastructure.

Warning: Value for undeclared variable

The root module does not declare a variable named "monitoring" but a value was found in file
"terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR_... environment variables to provide certain "global" settings to all
configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings
option.

Warning: Value for undeclared variable

The root module does not declare a variable named "subnet_count" but a value was found in file
"terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR_... environment variables to provide certain "global" settings to all
configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings
option.

Warning: Values for undeclared variables

In addition to the other similar warnings shown, 6 other variable(s) defined without being declared.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
optional_tag = {
  "Custom" = "dev"
  "Name" = "web-server"
}
value_received = {
  "cpu" = 4
  "name" = "server"
}
@manahilhabib031 → /workspaces/Lab9 (main) $ |

```

e) Null

Change terraform.tfvars:

dynamic_value = null

Run:

terraform apply -auto-approve

```
@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve
```

Changes to Outputs:

```
- value_received = {
  - cpu = 4
  - name = "server"
} -> null
```

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Warning: Value for undeclared variable

The root module does not declare a variable named "project_name" but a value was found in file "terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR_... environment variables to provide certain "global" settings to all configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings option.

Warning: Value for undeclared variable

The root module does not declare a variable named "monitoring" but a value was found in file "terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR_... environment variables to provide certain "global" settings to all configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings option.

Warning: Values for undeclared variables

In addition to the other similar warnings shown, 6 other variable(s) defined without being declared.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

```
optional_tag = {
  "Custom" = "dev"
  "Name" = "web-server"
}
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

Task 7 — Git ignore

Create .gitignore:

touch .gitignore

Add entries:

.terraform/*

*.tfstate

.tfstate.

*.tfvars

*.pem

```

@manahilhabib031 → /workspaces/Lab9 (main) $ touch .gitignore
@manahilhabib031 → /workspaces/Lab9 (main) $ vim .gitignore
@manahilhabib031 → /workspaces/Lab9 (main) $ cat .gitignore
.terraform/*
*.tfstate
*.tfstate.*
*.tfvars
*.pem

@manahilhabib031 → /workspaces/Lab9 (main) $ |

```

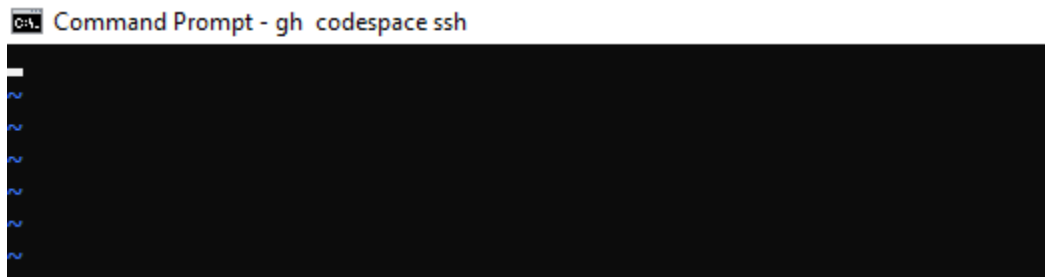
Task 8 — Clean-up then build real infra (VPC, Subnet, IGW, routing, default route table)

In this task you will clean previous example values and build a simple VPC + Subnet, attach an Internet Gateway, and configure routing (first with a custom route table and association, then switch to the default route table).

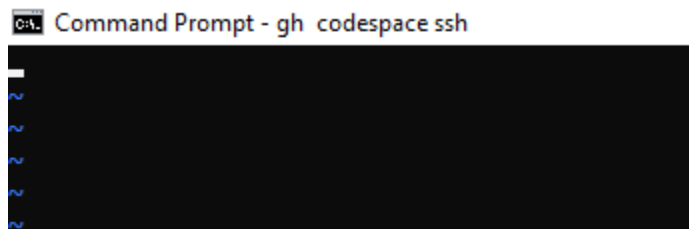
Perform all commands inside your Codespace shell.

Clean previous files

Remove all variable assignments from terraform.tfvars (empty the file or delete it).



Remove all content from locals.tf (delete or empty the file).



Replace main.tf contents with only the provider block below (start fresh).

Provider block to put in main.tf:

```

provider "aws" {
  shared_config_files = ["~/.aws/config"]
}

```

```
shared_credentials_files = ["~/.aws/credentials"]  
}
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ vim terraform.tfvars  
@manahilhabib031 → /workspaces/Lab9 (main) $ vim locals.tf  
@manahilhabib031 → /workspaces/Lab9 (main) $ vim main.tf  
@manahilhabib031 → /workspaces/Lab9 (main) $ cat > main.tf <<EOL  
provider "aws" {  
    shared_config_files      = ["~/.aws/config"]  
    shared_credentials_files = ["~/.aws/credentials"]  
}  
EOL  
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

Define variables in main.tf Add these variable declarations to main.tf (below the provider block):

```
variable "vpc_cidr_block" {}
```

```
variable "subnet_cidr_block" {}
```

```
variable "availability_zone" {}
```

```
variable "env_prefix" {}
```

```
Command Prompt - gh codespace ssh  
provider "aws" {  
    shared_config_files      = ["~/.aws/config"]  
    shared_credentials_files = ["~/.aws/credentials"]  
}  
variable "vpc_cidr_block" {}  
variable "subnet_cidr_block" {}  
variable "availability_zone" {}  
variable "env_prefix" {}
```

Create VPC in main.tf Add the VPC resource to main.tf:

```
resource "aws_vpc" "myapp_vpc" {  
    cidr_block = var.vpc_cidr_block  
    tags = {  
        Name = "${var.env_prefix}-vpc"  
    }  
}
```

Command Prompt - gh codespace ssh

```
provider "aws" {  
  shared_config_files    = ["~/.aws/config"]  
  shared_credentials_files = ["~/.aws/credentials"]  
}  
  
variable "vpc_cidr_block" {}  
variable "subnet_cidr_block" {}  
variable "availability_zone" {}  
variable "env_prefix" {}  
  
resource "aws_vpc" "myapp_vpc" {  
  cidr_block = var.vpc_cidr_block  
  
  tags = {  
    Name = "${var.env_prefix}-vpc"  
  }  
}
```

Create Subnet in the VPC Add the subnet resource to main.tf:

```
resource "aws_subnet" "myapp_subnet_1" {  
  vpc_id      = aws_vpc.myapp_vpc.id  
  cidr_block   = var.subnet_cidr_block  
  availability_zone = var.availability_zone  
  tags = {  
    Name = "${var.env_prefix}-subnet-1"  
  }  
}
```

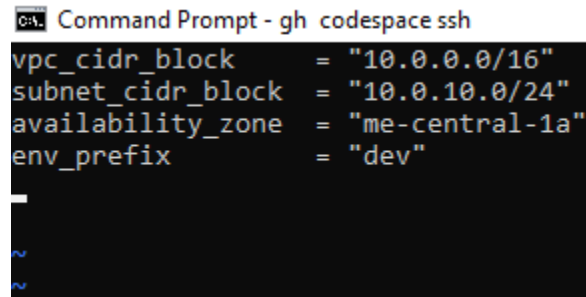
```
}  
  
resource "aws_subnet" "myapp_subnet_1" {  
  vpc_id      = aws_vpc.myapp_vpc.id  
  cidr_block   = var.subnet_cidr_block  
  availability_zone = var.availability_zone  
  tags = {  
    Name = "${var.env_prefix}-subnet-1"  
  }  
}
```

Populate terraform.tfvars In terraform.tfvars add:

```
vpc_cidr_block    = "10.0.0.0/16"  
subnet_cidr_block = "10.0.10.0/24"
```

availability_zone = "me-central-1a"

env_prefix = "dev"



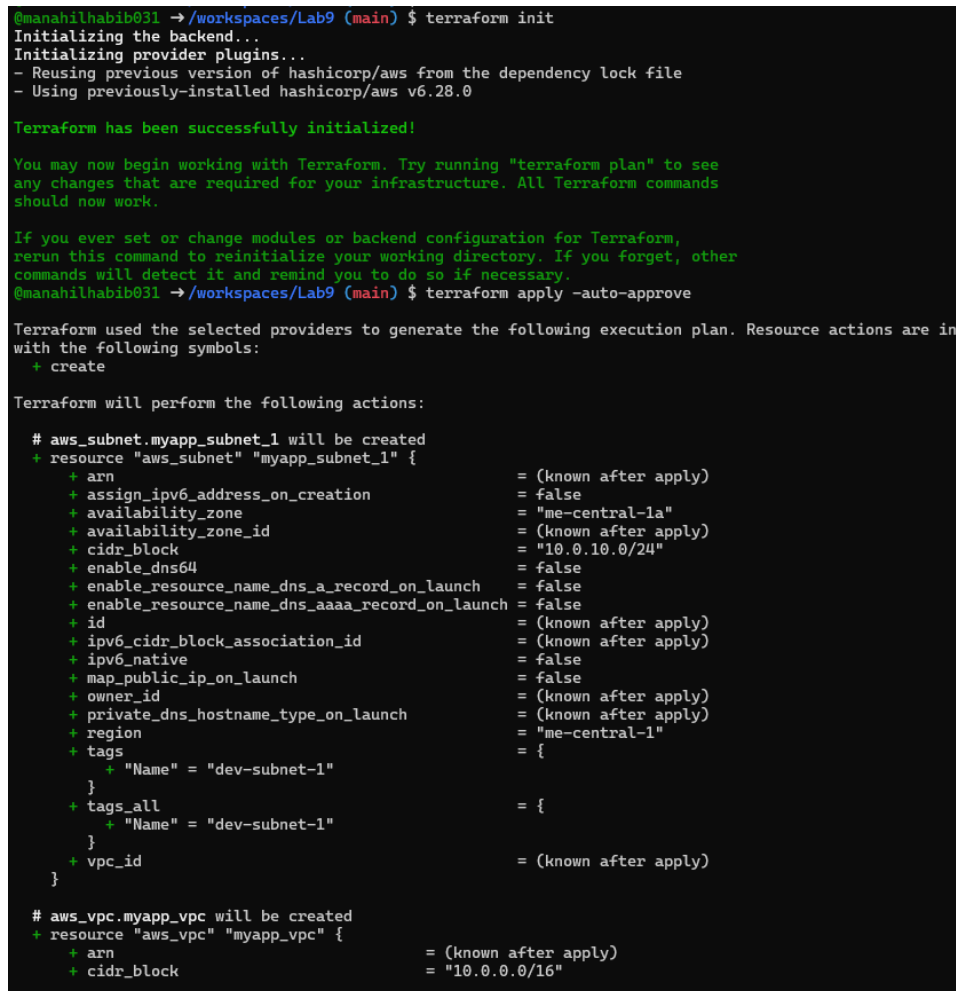
```
Command Prompt - gh codespace ssh
vpc_cidr_block      = "10.0.0.0/16"
subnet_cidr_block   = "10.0.10.0/24"
availability_zone    = "me-central-1a"
env_prefix          = "dev"
```

Apply to create VPC and Subnet Initialize (if needed) and apply:

terraform init

terraform apply -auto-approve

Verify in AWS Console that the VPC and Subnet were created.



```
@manahilhabib031 → /workspaces/Lab9 (main) $ terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.28.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve

Terraform used the selected providers to generate the following execution plan. Resource actions are in
with the following symbols:
+ create

Terraform will perform the following actions:

# aws_subnet.myapp_subnet_1 will be created
+ resource "aws_subnet" "myapp_subnet_1" {
  + arn                                = (known after apply)
  + assign_ipv6_address_on_creation    = false
  + availability_zone                  = "me-central-1a"
  + availability_zone_id               = (known after apply)
  + cidr_block                         = "10.0.10.0/24"
  + enable_dns64                      = false
  + enable_resource_name_dns_a_record_on_launch = false
  + enable_resource_name_dns_aaaa_record_on_launch = false
  + id                                = (known after apply)
  + ipv6_cidr_block_association_id     = (known after apply)
  + ipv6_native                       = false
  + map_public_ip_on_launch           = false
  + owner_id                          = (known after apply)
  + private_dns_hostname_type_on_launch = (known after apply)
  + region                            = "me-central-1"
  + tags                              = {
    + "Name" = "dev-subnet-1"
  }
  + tags_all                          = {
    + "Name" = "dev-subnet-1"
  }
  + vpc_id                            = (known after apply)
}

# aws_vpc.myapp_vpc will be created
+ resource "aws_vpc" "myapp_vpc" {
  + arn                = (known after apply)
  + cidr_block          = "10.0.0.0/16"
```

```

# aws_vpc.myapp_vpc will be created
+ resource "aws_vpc" "myapp_vpc" {
  + arn                = (known after apply)
  + cidr_block         = "10.0.0.0/16"
  + default_network_acl_id = (known after apply)
  + default_route_table_id = (known after apply)
  + default_security_group_id = (known after apply)
  + dhcp_options_id     = (known after apply)
  + enable_dns_hostnames = (known after apply)
  + enable_dns_support   = true
  + enable_network_address_usage_metrics = (known after apply)
  + id                  = (known after apply)
  + instance_tenancy     = "default"
  + ipv6_association_id   = (known after apply)
  + ipv6_cidr_block       = (known after apply)
  + ipv6_cidr_block_network_border_group = (known after apply)
  + main_route_table_id   = (known after apply)
  + owner_id              = (known after apply)
  + region                = "me-central-1"
  + tags                  = {
    + "Name" = "dev-vpc"
  }
  + tags_all              = {
    + "Name" = "dev-vpc"
  }
}

```

```

@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-0d51b63e5758a4576]
aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-0baba7cb6e234d1e1]

```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

```

+ create

```

Terraform will perform the following actions:

```

# aws_internet_gateway.myapp_igw will be created
+ resource "aws_internet_gateway" "myapp_igw" {
  + arn                = (known after apply)
  + id                 = (known after apply)
  + owner_id           = (known after apply)
  + region             = "me-central-1"
  + tags               = {
    + "Name" = "dev-igw"
  }
  + tags_all           = {
    + "Name" = "dev-igw"
  }
  + vpc_id             = "vpc-0d51b63e5758a4576"
}

# aws_route_table.myapp_route_table will be created
+ resource "aws_route_table" "myapp_route_table" {
  + arn                = (known after apply)
  + id                 = (known after apply)
  + owner_id           = (known after apply)
  + propagating_vgws    = (known after apply)
  + region             = "me-central-1"
  + route              = [
    + {
      + cidr_block         = "0.0.0.0/0"
      + gateway_id         = (known after apply)
      # (11 unchanged attributes hidden)
    },
  ]
  + tags               = {
    + "Name" = "dev-rt"
  }
  + tags_all           = {
    + "Name" = "dev-rt"
  }
  + vpc_id             = "vpc-0d51b63e5758a4576"
}

```

Plan: 2 to add, 0 to change, 0 to destroy.

```

aws_internet_gateway.myapp_igw: Creating...
aws_internet_gateway.myapp_igw: Creation complete after 0s [id=igw-0161adc8140cec108]

```

```

aws_route_table.myapp_route_table: Creating...

```

```

aws_route_table.myapp_route_table: Creation complete after 1s [id=rtb-07fd84b50553dc40e]

```

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

The screenshot displays the AWS Management Console interface for VPCs and Subnets. The top section, 'Your VPCs', shows two VPCs: 'dev-vpc' with ID 'vpc-0ec5b3b432d9d5ea9' and an unnamed VPC with ID 'vpc-006fd04ad83bb120a'. Both are in an 'Available' state. The bottom section, 'Subnets (4)', shows four subnets: three unnamed subnets and 'dev-subnet-1'. All subnets are in an 'Available' state and are associated with VPCs.

Name	VPC ID	State	Encryption c
dev-vpc	vpc-0ec5b3b432d9d5ea9	Available	-
-	vpc-006fd04ad83bb120a	Available	-

Name	Subnet ID	State	VPC
-	subnet-0bf5c35a7b6277b37	Available	vpc-006f
-	subnet-0dd7cbde7e97e05c8	Available	vpc-006f
-	subnet-022c8fa1a2d207ef6	Available	vpc-006f
dev-subnet-1	subnet-0ae659b6c3d5d723c	Available	vpc-0ec5

Create Internet Gateway and Route Table (custom) Add the Internet Gateway and a custom Route Table to main.tf:

```
resource "aws_internet_gateway" "myapp_igw" {
  vpc_id = aws_vpc.myapp_vpc.id
  tags = {
    Name = "${var.env_prefix}-igw"
  }
}

resource "aws_route_table" "myapp_route_table" {
  vpc_id = aws_vpc.myapp_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }
}
```

```

    }
    tags = {
      Name = "${var.env_prefix}-rt"
    }
  }
}

```

```

    }
  }
  resource "aws_internet_gateway" "myapp_igw" {
    vpc_id = aws_vpc.myapp_vpc.id
    tags = {
      Name = "${var.env_prefix}-igw"
    }
  }

  resource "aws_route_table" "myapp_route_table" {
    vpc_id = aws_vpc.myapp_vpc.id

    route {
      cidr_block = "0.0.0.0/0"
      gateway_id = aws_internet_gateway.myapp_igw.id
    }

    tags = {
      Name = "${var.env_prefix}-rt"
    }
  }
-- INSERT --

```

Apply:

```
terraform apply -auto-approve
```

Verify IGW and route table in AWS Console.

```

@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-0d51b63e5758a4576]
aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-0161adc8140cec108]
aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-0baba7cb6e234d1e1]
aws_route_table.myapp_route_table: Refreshing state... [id=rtb-07fd84b50553dc40e]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
  + create

Terraform will perform the following actions:

# aws_route_table_association.a_rtb_subnet will be created
+ resource "aws_route_table_association" "a_rtb_subnet" {
  + id            = (known after apply)
  + region        = "me-central-1"
  + route_table_id = "rtb-07fd84b50553dc40e"
  + subnet_id     = "subnet-0baba7cb6e234d1e1"
}

Plan: 1 to add, 0 to change, 0 to destroy.
aws_route_table_association.a_rtb_subnet: Creating...
aws_route_table_association.a_rtb_subnet: Creation complete after 0s [id=rtbassoc-0597a4621b34e36a2]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
@manahilhabib031 → /workspaces/Lab9 (main) $

```

Route tables (2) Info Last updated 8 minutes ago Actions Create route table

Find route tables by attribute or tag

☐	Name	Route table ID	Explicit subnet associ...	Edge
☐	-	rtb-0ef630eb168b8c51b	-	-
☐	-	rtb-0f9ed397121df91ee	-	-

Internet gateways (2) Info Actions Create internet gateway

Find internet gateways by attribute or tag

☐	Name	Internet gateway ID	State	VP
☐	-	igw-04ea345d94843568f	Attached	vp
☐	dev-igw	igw-0a885333769227a73	Attached	vp

Associate the Route Table with the Subnet Add the association resource to main.tf:

```

resource "aws_route_table_association" "a_rtb_subnet" {
  subnet_id    = aws_subnet.myapp_subnet_1.id
  route_table_id = aws_route_table.myapp_route_table.id
}

```

```

}

resource "aws_route_table_association" "a_rtb_subnet" {
  subnet_id      = aws_subnet.myapp_subnet_1.id
  route_table_id = aws_route_table.myapp_route_table.id
}

-- INSERT --

```

Apply:

terraform apply -auto-approve

Verify association in AWS Console (Subnet → Route Table).

```

@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-0d51b63e5758a4576]
aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-0161adc8140cec108]
aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-0baba7cb6e234d1e1]
aws_route_table.myapp_route_table: Refreshing state... [id=rtb-07fd84b50553dc40e]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
  + create

Terraform will perform the following actions:

# aws_route_table_association.a_rtb_subnet will be created
+ resource "aws_route_table_association" "a_rtb_subnet" {
  + id            = (known after apply)
  + region       = "me-central-1"
  + route_table_id = "rtb-07fd84b50553dc40e"
  + subnet_id    = "subnet-0baba7cb6e234d1e1"
}

Plan: 1 to add, 0 to change, 0 to destroy.
aws_route_table_association.a_rtb_subnet: Creating...
aws_route_table_association.a_rtb_subnet: Creation complete after 0s [id=rtbassoc-0597a4621b34e36a2]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
@manahilhabib031 → /workspaces/Lab9 (main) $ |

```

Route tables (1) Info					
Find route tables by attribute or tag Route table ID : rtb-0ef630eb168b8c51b Clear filters					
☐	Name	Route table ID	Explicit subnet associ...	Edge associations	Main
☐	-	rtb-0ef630eb168b8c51b	-	-	Yes

Switch to default route table (use VPC default route table) Now remove (or comment out) the custom route table and association resources from main.tf:

Remove the aws_route_table "myapp_route_table" block.

Remove the aws_route_table_association "a_rtb_subnet" block.

Then add this resource to update the default route table with a route to the IGW:

```

resource "aws_default_route_table" "main_rt" {

  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id

```

```

route {
  cidr_block = "0.0.0.0/0"
  gateway_id = aws_internet_gateway.myapp_igw.id
}

```

```

tags = {
  Name = "${var.env_prefix}-rt"
}
}

```

```

resource "aws_subnet" "myapp_subnet_1" {
  vpc_id            = aws_vpc.myapp_vpc.id
  cidr_block        = var.subnet_cidr_block
  availability_zone = var.availability_zone

  tags = {
    Name = "${var.env_prefix}-subnet-1"
  }
}
resource "aws_internet_gateway" "myapp_igw" {
  vpc_id = aws_vpc.myapp_vpc.id
  tags = {
    Name = "${var.env_prefix}-igw"
  }
}
resource "aws_default_route_table" "main_rt" {
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }

  tags = {
    Name = "${var.env_prefix}-rt"
  }
}

```

Apply:

terraform apply -auto-approve

Verify in AWS Console that the VPC's default route table has the 0.0.0.0/0 → IGW route and that the custom route table (if previously created) is no longer in use.

```
@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-0d51b63e5758a4576]
aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-0161adc8140cec108]
aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-0baba7cb6e234d1e1]
aws_route_table.myapp_route_table: Refreshing state... [id=rtb-07fd84b50553dc40e]
aws_route_table_association.a_rtb_subnet: Refreshing state... [id=rtbassoc-0597a4621b34e36a2]

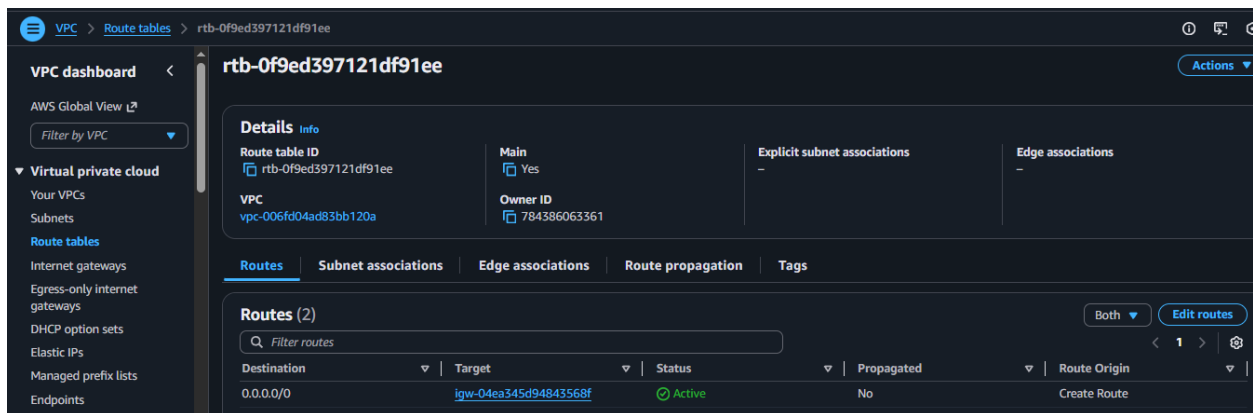
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
  + create

Terraform will perform the following actions:

# aws_default_route_table.main_rt will be created
+ resource "aws_default_route_table" "main_rt" {
  + arn                = (known after apply)
  + default_route_table_id = "rtb-013cb9c66253a4c60"
  + id                 = (known after apply)
  + owner_id           = (known after apply)
  + region             = "me-central-1"
  + route              = [
    + {
      + cidr_block      = "0.0.0.0/0"
      + gateway_id      = "igw-0161adc8140cec108"
      # (10 unchanged attributes hidden)
    },
  ]
  + tags               = {
    + "Name" = "dev-rt"
  }
  + tags_all           = {
    + "Name" = "dev-rt"
  }
  + vpc_id             = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.
aws_default_route_table.main_rt: Creating...
aws_default_route_table.main_rt: Creation complete after 1s [id=rtb-013cb9c66253a4c60]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```



Task 9 — Security Group, Key Pair, EC2 Instance, user data & nginx

This task walks you through creating a security group, creating an EC2 key pair, launching an EC2 instance, verifying SSH access, and installing nginx via user_data (inline and from a script). Perform all commands from your Codespace shell.

Add variables to main.tf Add these variables to your main.tf:

```
variable "my_ip" {}
```

```
}  
}  
variable "my_ip" {}  
-- REPLACE --
```

Get your public IP and set terraform.tfvars From the Codespace shell:

```
curl icanhazip.com
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ vim main.tf  
@manahilhabib031 → /workspaces/Lab9 (main) $ curl icanhazip.com  
4.240.39.196  
@manahilhabib031 → /workspaces/Lab9 (main) $ vim terraform.tfvars  
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

Copy the IP and add to terraform.tfvars:

```
my_ip = "<your_ip>/32"
```

```
instance_type = "t3.micro"
```

```
availability_zone = "me-central-1a" # or your chosen AZ
```

```
env_prefix = "dev"
```

```
my_ip = "4.240.18.230/32"  
instance_type = "t3.micro"  
availability_zone = "me-central-1a"  
env_prefix = "dev"
```

Create the Security Group (main.tf) Add this resource to main.tf:

```
resource "aws_default_security_group" "myapp_sg" {
```

```
  vpc_id    = aws_vpc.myapp_vpc.id
```

```
  ingress {
```

```
    from_port = 22
```

```
    to_port   = 22
```

```
    protocol  = "tcp"
```

```
    cidr_blocks = [var.my_ip]
```

```
  }
```

```
  ingress {
```

```
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
}
egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
    prefix_list_ids = []
}
tags = {
    Name = "${var.env_prefix}-sg"
}
}
```

```
0% Command Prompt - gh codespace ssh
resource "aws_default_route_table" "main_rt" {
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }

  tags = {
    Name = "${var.env_prefix}-rt"
  }
}
variable "my_ip" {}
resource "aws_default_security_group" "myapp_sg" {
  vpc_id = aws_vpc.myapp_vpc.id

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = [var.my_ip]
  }

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "${var.env_prefix}-sg"
  }
}
```

Run apply:

terraform apply -auto-approve

```

@manahilhabib031 → /workspaces/Lab9 (main) $ terraform apply -auto-approve
aws_route_table_association.a_rtb_subnet: Refreshing state... [id=rtbassoc-0597a4621b34e36a2]
aws_default_route_table.main_rt: Refreshing state... [id=rtb-013cb9c66253a4c60]
aws_route_table.myapp_route_table: Refreshing state... [id=rtb-07fd84b50553dc40e]
aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-0baba7cb6e234d1e1]
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-0d51b63e5758a4576]
aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-0161adc8140cec108]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
+ create
- destroy

Terraform planned the following actions, but then encountered a problem:

# aws_default_route_table.main_rt will be destroyed
# (because aws_default_route_table.main_rt is not in configuration)
- resource "aws_default_route_table" "main_rt" {
  - arn = "arn:aws:ec2:me-central-1:095032391730:route-table/rtb-013cb9c66253a4c60" -> null
}

  - default_route_table_id = "rtb-013cb9c66253a4c60" -> null
  - id = "rtb-013cb9c66253a4c60" -> null
  - owner_id = "095032391730" -> null
  - propagating_vgws = [] -> null
  - region = "me-central-1" -> null
  - route = [
    - {
      - cidr_block = "0.0.0.0/0"
      - gateway_id = "igw-0161adc8140cec108"
      # (10 unchanged attributes hidden)
    },
  ] -> null
  - tags = {
    - "Name" = "dev-rt"
  } -> null
  - tags_all = {
    - "Name" = "dev-rt"
  } -> null
  - vpc_id = "vpc-0d51b63e5758a4576" -> null
}

# aws_key_pair.ssh_key will be created
+ resource "aws_key_pair" "ssh_key" {
+ arn = (known after apply)
+ fingerprint = (known after apply)
}

```

Create an AWS key pair and save locally Create a key pair and store the private key in your Codespace. Do NOT commit the .pem file.

```
aws ec2 create-key-pair \
```

```
--key-name MyED25519Key \
```

```
--key-type ed25519 \
```

```
--key-format pem \
```

```
--query 'KeyMaterial' \
```

```
--output text > MyED25519Key.pem
```

```
chmod 600 MyED25519Key.pem
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 create-key-pair \
  --key-name MyED25519Key \
  --key-type ed25519 \
  --key-format pem \
  --query 'KeyMaterial' \
  --output text > MyED25519Key.pem
-bash: MyED25519Key.pem: Permission denied
@manahilhabib031 → /workspaces/Lab9 (main) $ chmod 600 MyED25519Key.pem
```

Ensure .gitignore contains:

*.pem

Add EC2 instance resource (initial) Add the instance resource to main.tf (initially using the created key name):

```
resource "aws_instance" "myapp-server" {
  ami           = "ami-05524d6658fcf35b6" # Amazon Linux 2023
  instance_type = var.instance_type
  subnet_id     = aws_subnet.myapp_subnet_1.id
  security_groups = [aws_default_security_group.default_sg.id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name       = "MyED25519Key"
  tags = {
    Name = "${var.env_prefix}-ec2-instance"
  }
}

output "aws_instance_public_ip" {
  value = aws_instance.myapp-server.public_ip
}
```

```

resource "aws_instance" "myapp-server" {
  ami           = "ami-05524d6658fcf35b6"
  instance_type = var.instance_type
  subnet_id     = aws_subnet.myapp_subnet_1.id
  security_groups = [aws_default_security_group.myapp_sg.id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name       = "MyED25519Key"

  tags = {
    Name = "${var.env_prefix}-ec2-instance"
  }
}

output "aws_instance_public_ip" {
  value = aws_instance.myapp-server.public_ip
}

-- INSERT --

```

Run:

terraform apply -auto-approve

```

aws_vpc.myapp_vpc: Refreshing state... [id=vpc-0ec5b3b432d9d5ea9]
aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-0ae659b6c3d5d723c]
aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-0a885333769227a73]
aws_default_security_group.myapp_sg: Refreshing state... [id=sg-020e559e3a639f5dc]
aws_default_route_table.main_rt: Refreshing state... [id=rtb-0ef630eb168b8c51b]

Terraform used the selected providers to generate the following execution plan. Resource
+ create

Terraform will perform the following actions:

# aws_instance.myapp-server will be created
+ resource "aws_instance" "myapp-server" {
  + ami           = "ami-05524d6658fcf35b6"
  + arn           = (known after apply)
  + associate_public_ip_address = true
  + availability_zone = "me-central-1a"
  + disable_api_stop = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized   = (known after apply)
  + enable_primary_ipv6 = (known after apply)
  + force_destroy   = false
}

```

```

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ aws_instance_public_ip = (known after apply)
aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Creation complete after 13s [id=i-016dc03e800f78406]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:
aws_instance_public_ip = "40.172.100.251"

```

SSH into the instance (using MyED25519Key) From the Codespace:

ssh -i MyED25519Key.pem ec2-user@<public-ip>

verify commands, then exit

Exit

```
@manahilhabib031 → /workspaces/Lab9 (main) $ ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""
Generating public/private ed25519 key pair.
/home/codespace/.ssh/id_ed25519 already exists.
Overwrite (y/n)? y
Your identification has been saved in /home/codespace/.ssh/id_ed25519
Your public key has been saved in /home/codespace/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:9/DFH3mrMyR7gLWxuqMLFCN5VI0dY+S287/9/RCj2aI codespace@codespaces-f6837e
The key's randomart image is:
+--[ED25519 256]--+
|      .+=      |
|      o..=      |
|      o+.o.     |
|      o+.o. .   |
|      .So= + =o. |
|      . ooO * ++ |
|      . ...@ o.. |
|      oo o.=o..  |
|      ...E .++.* |
+-----[SHA256]-----+
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

Generate a local SSH keypair and register it in AWS via Terraform On your Codespace, generate an SSH key pair (accept defaults or specify path):

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""
```

```
@23-22411-071-eng → /workspaces/my-lab-repo (main) $ ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""
Generating public/private ed25519 key pair.
Your identification has been saved in /home/codespace/.ssh/id_ed25519
Your public key has been saved in /home/codespace/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:AdLIUDtBJEUjqNBij7ydgrMaOZTp+Rw9rj2Fukn9Vn4 codespace@codespaces-d737cf
The key's randomart image is:
+--[ED25519 256]--+
| o+OXo.         |
| +..oo=..       |
| =.o o .        |
| .oo. . .       |
| .+o . . S      |
| =+.oo. . .     |
| +=.o.+ o       |
| .o+.=.o. . E   |
| o *ooo. .      |
+-----[SHA256]-----+
@23-22411-071-eng → /workspaces/my-lab-repo (main) $
```

Add a Terraform resource in main.tf to register the public key:

```
resource "aws_key_pair" "ssh_key" {
  key_name = "serverkey"
  public_key = file("~/ssh/id_ed25519.pub")
}
```

Update the EC2 resource to use the Terraform-managed key:

```
resource "aws_instance" "myapp-server" {  
  ...  
  # replace key_name = "MyED25519Key" with:  
  key_name = aws_key_pair.ssh_key.key_name  
  ...  
}
```

```
resource "aws_instance" "myapp-server" {  
  ami                = "ami-05524d6658fcf35b6"  
  instance_type      = var.instance_type  
  subnet_id          = aws_subnet.myapp_subnet_1.id  
  security_groups    = [aws_default_security_group.myapp_sg.id]  
  availability_zone   = var.availability_zone  
  associate_public_ip_address = true  
  key_name            = aws_key_pair.ssh_key.key_name  
  tags = {  
    Name = "${var.env_prefix}-ec2-instance"  
  }  
}  
  
resource "aws_key_pair" "ssh_key" {  
  key_name    = "serverkey"  
  public_key = file("~/ssh/id_ed25519.pub")  
}  
~  
~  
~  
~  
-- INSERT --
```

Run:

```
terraform apply -auto-approve
```

```

g23-22411-071-eng @ /workspaces/my-lab-repo (main) $ terraform apply -auto-approve
aws_key_pair.ssh_key: Refreshing state... [id=serverkey]
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-0ec5b3b432d9d5ea9]
aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-0a885333769227a73]
aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-0ae659b6c3d5d723c]
aws_default_security_group.myapp_sg: Refreshing state... [id=sg-020e559e3a639f5dc]
aws_default_route_table.main_rt: Refreshing state... [id=rtb-0ef630eb168b8c51b]

Terraform used the selected providers to generate the following execution plan. Resources to be
created:
+ create

Terraform will perform the following actions:

# aws_instance.myapp-server will be created
+ resource "aws_instance" "myapp-server" {
+   ami                    = "ami-05524d6658fcf35b6"
+   arn                   = (known after apply)
+   associate_public_ip_address = true
+   availability_zone      = "me-central-1a"
+   disable_api_stop       = (known after apply)
+   disable_api_termination = (known after apply)
+   ebs_optimized          = (known after apply)
+   enable_primary_ipv6    = (known after apply)
+   force_destroy          = false
+   get_password_data       = false
+   host_id                = (known after apply)
+   host_resource_group_arn = (known after apply)
+   iam_instance_profile    = (known after apply)
+   id                    = (known after apply)
+   instance_initiated_shutdown_behavior = (known after apply)
+   instance_lifecycle     = (known after apply)

```

```

+   tenancy                = (known after apply)
+   user_data_base64       = (known after apply)
+   user_data_replace_on_change = false
+   vpc_security_group_ids = (known after apply)

+   capacity_reservation_specification (known after apply)

+   cpu_options (known after apply)

+   ebs_block_device (known after apply)

+   enclave_options (known after apply)

+   ephemeral_block_device (known after apply)

+   instance_market_options (known after apply)

+   maintenance_options (known after apply)

+   metadata_options (known after apply)

+   network_interface (known after apply)

+   primary_network_interface (known after apply)

+   private_dns_name_options (known after apply)

+   root_block_device (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.
aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Creation complete after 13s [id=i-0f97

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

```

SSH using the newly registered key Now SSH with your generated private key (the default ssh client will pick up ~/.ssh/id_ed25519):

Verify login works.

```

[ec2-user@ip-10-0-10-30 ~]$ ssh ec2-user@40.172.113.48
The authenticity of host '40.172.113.48 (40.172.113.48)' can't be established.
ED25519 public key fingerprint is SHA256:1TWGX1pfeHfKw2Ql9t4nPghYh5+WalgUTj594J5X07A.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '40.172.113.48' (ED25519) to the list of known hosts.

#
~ \ ##### Amazon Linux 2023
~~ \ ##### \
~~ \ ### |
~~ \ #/ https://aws.amazon.com/linux/amazon-linux-2023
~~ \ ~ ' ->
    ~~~
    ~ ~ . - .
    ~ ~ / ' - /
    ~ ~ / m / ' - /
[ec2-user@ip-10-0-10-30 ~]$ whoami
ec2-user
[ec2-user@ip-10-0-10-30 ~]$ exit
logout
Connection to 40.172.113.48 closed.

```

Install nginx via inline user_data Modify the aws_instance resource to include inline user_data:

```
resource "aws_instance" "myapp-server" {
```

...

```
user_data = <<-EOF
```

```
#!/bin/bash
```

```
yum update -y
```

```
yum install -y nginx
```

```
systemctl start nginx
```

```
systemctl enable nginx
```

EOF

$$\text{tags} = \{$$

```
Name = "${var.env_prefix}-ec2-instance"
```

$$\}$$

}

```

resource "aws_instance" "myapp-server" {
  ami              = "ami-05524d6658fcf35b6"
  instance_type    = var.instance_type
  subnet_id        = aws_subnet.myapp_subnet_1.id
  security_groups  = [aws_default_security_group.myapp_sg.id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name         = aws_key_pair.ssh_key.key_name
  user_data = <<-EOF
    #!/bin/bash
    yum update -y
    yum install -y nginx
    systemctl start nginx
    systemctl enable nginx
  EOF

  tags = {
    Name = "${var.env_prefix}-ec2-instance"
  }
}

resource "aws_key_pair" "ssh_key" {
  key_name   = "serverkey"
  public_key = file("~/ssh/id_ed25519.pub")
}

output "aws_instance_public_ip" {
  value = aws_instance.myapp-server.public_ip
}

```

Apply:

terraform apply -auto-approve

```

- throughput      = 125 -> null
- volume_id       = "vol-02e138659312c5b98" -> null
- volume_size     = 8 -> null
- volume_type     = "gp3" -> null
  # (1 unchanged attribute hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  ~ aws_instance_public_ip = "40.172.113.48" -> (known after apply)
aws_instance.myapp-server: Destroying... [id=i-0ceed0667ab75b874]
aws_instance.myapp-server: Still destroying... [id=i-0ceed0667ab75b874, 00m10s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0ceed0667ab75b874, 00m20s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0ceed0667ab75b874, 00m30s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0ceed0667ab75b874, 00m40s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0ceed0667ab75b874, 00m50s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0ceed0667ab75b874, 01m00s elapsed]
aws_instance.myapp-server: Destruction complete after 1m0s
aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Creation complete after 12s [id=i-0ac0921a0004aa65a]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:
aws_instance_public_ip = "158.252.34.230"

```

SSH in and run:

curl localhost

```
The authenticity of host '158.252.34.230 (158.252.34.230)' can't be established.  
ED25519 key fingerprint is SHA256:/7mwu5LgOAKBxrAhuUfNDSam6wYXjnm+Yz2FR9FYOA4.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '158.252.34.230' (ED25519) to the list of known hosts.
```

Open `http://<public-ip>` in a browser — you should see the nginx default page.

Welcome to nginx!

Use an external script for user data Create a script file in the Codespace:

```
cat > entry-script.sh <<'EOF'
```

```
yum update -y
```

systemctl start nginx

systemctl enable nginx

EOF

```
@manahilhabib031 → /workspaces/Lab9 (main) $ cat > entry-script.sh <<'EOF'
#!/bin/bash
yum update -y
yum install -y nginx
systemctl start nginx
systemctl enable nginx
EOF
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

Update aws_instance to use the file:

```
resource "aws_instance" "myapp-server" {
  ...
  user_data = file("entry-script.sh")
  tags = {
    Name = "${var.env_prefix}-ec2-instance"
  }
}
```

```
}
resource "aws_instance" "myapp-server" {
  ami                = "ami-05524d6658fcf35b6"
  instance_type      = var.instance_type
  subnet_id          = aws_subnet.myapp_subnet_1.id
  security_groups    = [aws_default_security_group.id]
  availability_zone   = var.availability_zone
  associate_public_ip_address = true
  key_name            = aws_key_pair.ssh_key.key_name
  user_data           = file("entry-script.sh")
  tags = {
    Name = "${var.env_prefix}-ec2-instance"
  }
}

resource "aws_key_pair" "ssh_key" {
```

Apply:

terraform apply -auto-approve

```

- throughput          = 125 -> null
- volume_id           = "vol-0f2e59989468913d3" -> null
- volume_size         = 8 -> null
- volume_type         = "gp3" -> null
  # (1 unchanged attribute hidden)
}
}

Plan: 1 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  ~ aws_instance_public_ip = "158.252.34.230" -> (known after apply)
aws_instance.myapp-server: Destroying... [id=i-0ac0921a0004aa65a]
aws_instance.myapp-server: Still destroying... [id=i-0ac0921a0004aa65a, 00m10s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0ac0921a0004aa65a, 00m20s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0ac0921a0004aa65a, 00m30s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0ac0921a0004aa65a, 00m40s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0ac0921a0004aa65a, 00m50s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0ac0921a0004aa65a, 01m00s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0ac0921a0004aa65a, 01m10s elapsed]
aws_instance.myapp-server: Destruction complete after 1m10s
aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Creation complete after 13s [id=i-04362e050f30c0ddf]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:
aws_instance_public_ip = "3.28.46.221"

```

Reload the browser at <http://<public-ip>> to confirm nginx is served.

3.28.46.221

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Cleanup

Destroy all resources:

terraform destroy -auto-approve

Verify state files:

cat terraform.tfstate

```
@manahilhabib031 → /workspaces/Lab9 (main) $ terraform destroy -auto-approve
aws_default_route_table.main_rt: Refreshing state... [id=rtb-013cb9c66253a4c60]
aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-0baba7cb6e234d1e1]
aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-0161adc8140cec108]
aws_route_table_association.a_rt_b_subnet: Refreshing state... [id=rtbassoc-0597a4621b34e36a2]
aws_route_table.myapp_route_table: Refreshing state... [id=rtb-07fd84b50553dc40e]
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-0d51b63e5758a4576]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
  - destroy

Terraform will perform the following actions:

# aws_default_route_table.main_rt will be destroyed
- resource "aws_default_route_table" "main_rt" {
  - arn              = "arn:aws:ec2:me-central-1:095032391730:route-table/rtb-013cb9c66253a4c60" -> nul
l
  - default_route_table_id = "rtb-013cb9c66253a4c60" -> null
  - id                    = "rtb-013cb9c66253a4c60" -> null
  - owner_id              = "095032391730" -> null
  - propagating_vgws      = [] -> null
  - region                = "me-central-1" -> null
  - route                 = [

```

cat terraform.tfstate.backup

```
Destroy complete! Resources: 0 destroyed.
@manahilhabib031 → /workspaces/Lab9 (main) $ cat terraform.tfstate
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 84,
  "lineage": "8b1d5d5f-e9bc-aaf3-332c-e079db2b49e2",
  "outputs": {},
  "resources": [],
  "check_results": null
}
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

```

@manahilhabib031 → /workspaces/Lab9 (main) $ cat terraform.tfstate.backup
{
  "version": 4,
  "terraform_version": "1.14.3",
  "serial": 77,
  "lineage": "8b1d5d5f-e9bc-aaf3-332c-e079db2b49e2",
  "outputs": {},
  "resources": [
    {
      "mode": "managed",
      "type": "aws_default_route_table",
      "name": "main_rt",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        {
          "schema_version": 0,
          "attributes": {
            "arn": "arn:aws:ec2:me-central-1:095032391730:route-table/rtb-013cb9c66253a4c60",
            "default_route_table_id": "rtb-013cb9c66253a4c60",
            "id": "rtb-013cb9c66253a4c60",
            "owner_id": "095032391730",
            "propagating_vgws": null,
            "region": "me-central-1",
            "route": [
              {
                "cidr_block": "0.0.0.0/0",

```

Ensure no sensitive files are committed. If any private keys or credentials were accidentally committed — rotate them immediately.

```

@manahilhabib031 → /workspaces/Lab9 (main) $ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .gitignore
    .terraform.lock.hcl
    aws/
    awscli2.zip
    entry-script.sh
    locals.tf
    main.tf

nothing added to commit but untracked files present (use "git add" to track)
@manahilhabib031 → /workspaces/Lab9 (main) $ |

```

```

@manahilhabib031 → /workspaces/Lab9 (main) $ cat .gitignore
.terraform/*
*.tfstate
*.tfstate.*
*.tfvars
*.pem

```